Common

[+cmd] option
See :help +cmd
When editing a file, specify what to do after opening a file.
Usually which line to start
e.g.
 :buffer +10 myfile.txt start at line 10

+ last line
+10
+/hello first line matching pattern
+/h.llo
+/hello\ world escape whitespace

+set\ list any Ex command. Need to escape whitespace.
+set\ list\ |\ set\ number multiple commands

```
e.g.
:buffer +set\ list 3

" same
:buffer 3
:set list
```

[++opt] option
See :help ++opt
When editing a file, specifying inital file format related options
e.g.
  :edit ++enc=utf-8 myfile.txt

++enc=utf-8 Overwrite fileencodings (read encodings) and fileencoding (write encoding)
  See :help encoding-values
++ff=unix Overwrite fileformat (how to interpret new line). See :help ff
++ff=dos
++ff=mac
++bin set binary
++nobin

Buffer ranges
See :help range
Used in operations that can specify a range
e.g. bdelete
  :1,10bd delete buffer 1-10 inclusive

```
1               Buffer 1
.               Current
.+              One after current
.+2             Two after current
.-              One before current
$               Last
$-              One before last
1,10            Range inclusive. i.e. 1~10
1:10            Same
```

==========
======

Buffer

Keys

```
ZZ          Write and quit. Same as :wq
```

Buffer indicators

See ':help :ls'

Indicator
Type of buffer.
e.g.

```
:ls
  1  a   "src/api/view/service.js"      line 0
  4  a   "src/database/redis/index.js"  line 0
  5  a-  "[Location List]"              line 0
  7 %a   "src/utils/DistributedRecomputables.js" line 1
```

```
%    Buffer in current window
#    Alternate buffer. i.e. previously open
a    Active buffer. e.g. loaded in a tab
h    hidden buffer e.g. loaded before but not
%a   Current buffer.
u    Unlisted. Only listed with ls!.
-    Non-modifiable. i.e. no 'modifiable' flag. e.g.
Quickfix window
=    Readonly
```

Buffer name

See :help {bufname}

./documents/myfile.txt	Full name
myfile.txt	Partial name, as long as it is unique
myf	Same

Commands

```
q!       exit without saving

"" Edit
edit                    ' Reload current file
edit filename           ' Opens the given file.
edit ++enc=utf-8        ' Overwrite fileencodings and
fileencoding


"" read - Insert into current buffer
" Reads some content and inserts it under current
position
read myfile.txt          Insert content of myfile.txt
r myfile.txt             Same
r ! ls                   Insert stdout of command.


"" write - Write current buffer
"
write                    Write buffer to current file
```

```
w                       Same
wq                      Write and quit
write newfile           Write to new file

write! newfile          Force write in readonly mode
write ! cat             Pass current buffer as stdin of
command


"" ls - See buffer list
"
"
ls          See buffer list
buffers     Same
files       Same
ls!         All buffers. Some buffers are not shown by
default (e.g. help)


"" badd
badd mybuf
bad mybuf
bad ./mybuf     " Same


"" bdelete
"" bwipeout
" Remove buffer.
" bdelete only removes it from buffer list.
" i.e. still kept in memory.
" bwipeout actually deletes it from memory.
bdelete            Remove current buffer
bd
bwipe              Remove current buffer
bw
e.g.
:bdelete 1
:buffers
  2 %a  "[No Name]"
:buffers!
  1u#    "myfile.txt"              myfile.txt still
exists
  2 %a  "[No Name]"

:bwipeout 1
:buffers!
  2 %a  "[No Name]"           myfile.txt removed

" By buffer number
bd 1               Remove buffer by number. i.e. 1
bw 1
bd ./myfile.txt    Remove buffer by bufname
bd myfile          Same
bd #               Remove by buffer indicator

" Delete range (see range specs above)
```

```
%bd                Delete all buffers
1bd                Same as 'bd 1'
1,10bd             Remove buffer 1-10 including 1 and 10
1,.-bd             Delete all before current window.


"" moving between buffers
"
" With buffer number
buffer 3       Edit buffer 3
b 3            Same
b3
" With bufname
b myfile.txt
b myf          Find closest match
b #

" With +cmd
b +10 3        With +cmd. Start at line 10.

" buffer in split window
sbuffer 3      Edit buffer 3 in horizontal split (below)
sb 3
vert sb 3                  Vertical split on left
vert belowright sb 3       Vertical split on right


"" bufdo  Run command on loaded buffers
" Similar to argdo
bufdo echo 'hello'
bufdo call system("touch a")          e.g. call function
bufdo write                           e.g. save all
buffers
bufdo %s/aaa/bbb/gc                   e.g. replace in
all buffers



bnext          Next buffer
bn             Same
bn 1           Same
bn 2           Two buffer after

bprevious      Previous buffer
bp
bp 1           Same
bp 2           Two buffer before

bfirst
blast


Functions
"" bufname(expr)
" Gets the absolute path of the buffer
bufname(1)              # Absolute path of buffer 1
```

```
 e.g. './myfile.txt'
bufname(100000)         # Empty string if buffer doesn't
exist. i.e. ''

" If expr is a string, it is used as a file-pattern.
bufname('file.txt')     # Absolute path of file.txt
bufname('%')            # path of current buffer
bufname('#')            # path of alternate buffer


"" bufnr(expr)
" Gets the buffer number of the current buffer
" e.g. 1
bufnr('file.txt')       # Buffer number of file.txt.
bufnr('randomfile')     # -1 if it does not exist
bufnr('%')              # Current buffer
bufnr('#')              # Alternate buffer
if bufnr('#') != -1     # if buffer exists
  …
endif


"" bufexists(expr)
" TRUE if
bufexists('file.txt')       # by name
bufexists('%')              # by expression
bufexists(1)                # by buffer number
if bufexist(1)
 …
endif

" etc
bufadd()
bufload()
buflisted()
bufloaded()
bufwinid()
bufwinnr()
```

==========

Argument list

argument list: a group of buffers
- not the same as buffers. Argument lists is just a light subgrouping of buffers.
- name 'argument' is historical; it was the arguments passed to vi
  e.g. vi a.txt b.txt

Commands

```
:help args

"" args
"
" Print/set argument list
```

```
:args
:args a.txt b.txt       " Set argument list to a.txt
b.txt
:args src/**/*.js       " Can use wildcard


" Modifying
"
" Add to argument list
:argadd c.txt
:arga c.txt

:argdelete c.txt
:argd c.txt


" Navigating
"
:argument 2         Edit argument 2
:argu 2

:next               Next argument
:n                  Same
:2n                 Next next

:previous
:prev               Same
:2prev              Previous previous

:rewind             First argument
:first              Same
:last               Last argument


"" argdo
"
" Run command in args
" Equivalent to
"   :rewind
"   :{cmd}
"   :next
"   :{cmd}
"   ...
"
argdo %s/hello/Hello/g           "
argdo %s/hello/Hello/g | w       " Multiple commands
argdo /Hello                     " Search in every file
for Hello
```

Misc

*) "E37 No change since last write" error on argdo
By default, argdo won't move to next buffer if not saved.
Need to write buffer after argdo

```
argdo s/hello/Hello/g       " ERROR
argdo s/hello/Hello/g | w   " OK
```

==========
======
Split

:help window.txt

:split *filename* Split horizontally.
:vsplit *filename* Split vertically.
:only Close all windows except the current one.
:close Closes the current window.

c-w s :split
c-w c-s
c-w v :vsplit
c-w c-v
c-w q Close window
c-w c-q

c-w = Make all window sizes equal
c-w_ Max height
10c-w_ 10 lines high
c-w - Shorter
10c-w - 10 lines shorter
c-w + Taller
10c-w + 10 lines taller

c-w| Max width
10c-w| 10 lines wide
c-w > Wider
10c-w > 10 lines wider
c-w < Narrower
10c-w < 10 lines narrower
:res -10 Make window 10 units lower.
:res +10 Make window 10 units higher.

c-w w Cycle through windows clockwise (from top left)
c-w W Anticlockwise
1c-w w To window #1 (first)
4c-w w To window #4
c-w p Previous window
c-w h Move cursor left
c-w k Move cursor up.
c-w j Move cursor down.
c-w l Move cursor right.
c-w t Top left window
c-w b Bottom right window

c-w H Move window far left
c-w J Top
c-w K Bottom
c-w L Right
c-w T To new tab

==========
======
Tabs

```
:help tabpage.txt
```

Command

```
:tabedit     file               Open new file in tab

:tabclose
:tabonly             Close all other tabs
:qa                  Close all tabs
:wqa                 Save and close all tabs

:tabsplit
:tabn
:tabp

:tabs                    List all tabs.
:tabm 2                  Move tab to after tab 2.
:tabm 0                  Move tab to first.
:tabm                    Move tab to end.

# In normal mode
gt          Next tab
gT         Previous tab
3gt        3rd tab
```

==========

======

Quickfix

:help quickfix.txt

Quickfix:
e.g. grep results

location list: Same as quickfix, but limited to current window
e.g. lint errors

Commands

```
"" cc[!]
" Jump to error
cc           " Jump to current error
cc!          "
cc 1         " Jump to first error
cc 1000      " ERROR if cannot jump

cfirst     " Same as cc 1
clast      " Jump to last error


"" cnext
" Jump to next error
cnext
1cnext     " Same
2cnext     "


"" cprevious
```

```
" Jump to previous error
cprevious
1cprevious
2cprevious
:lnext

"" clist[from [,to]]
" List errors. Current error is highlighted
clist
clist 1       " List first error only
clist 1,5     " Errors 1~5

llist +0      " List only current error
clist +1      " List current and nextjjj


"" cdo {cmd}
cdo normal @1
cdo normal @1 | normal @2     " concatenate commands


"" copen [height]
" Open current list of errors in a window
copen
copen 10         " Default
copen 3          " 3 lines hight


"" cclose
" Close copen'ed window
cclose
ccl


"" cwindow
" Same as copen, except the window is not opened if
there are no errors.
"  If the window is already open and there are no
errors, it is closed
cwindow
cw



" Location list equivalent
ll
lfirst
llast
lnext
lprevious
ldo
lopen
lclose
lwindow
```

Filtering

Ref: https://www.reddit.com/r/vim/comments/hbcx5d/
the_quickfix_and_location_lists_in_vim/

```
packadd cfilter

Cfilter /myfile/
Cfilter! /myfile/
Lfilter ...
```

```
 ========================================================
                                ======
```

Grep

:help vimgrep
:help grep

Vim has two types of grep: vimgrep and external grep command

Vimgrep

```
## vimgrep[!] /PATTERN/[g][j] files...
:vimgrep {regex} file.txt               Looks for the
given pattern in the given file.
:vimgrep {regex} file1.txt  file2.txt    Multiple
:vimgrep {regex}/i file.txt
:vimgrep {regex} *.txt                   all txt files
of the current directory.
:vimgrep {regex} **/*.txt                all txt files
of the current and all its subdirectories, down to depth
100.
:vimgrep {regex} comp**/*.txt
:cn     Next match.
:cp     Previous match.
:cw     List of all matches.

:help pattern-overview
:help vimgrep

" Examples
vimgrep =^\t= **/*.js          Grep for tab indentation
in all JS files
```

grep

```
"" grep[!] ARGS
" Run grep
:grep hello file.txt
:grep! hello file.txt          " Do not jump to first
match automatically
```

Misc

*) grep regex

```
# Extended regex
:grep -E 'a{2,}' file.txt

# Special case: need to escape |
:grep -E '(a\|b)' file.txt
```

==========

Netrw

:help netrw

Keys

In the file explorer

-	Move up one directory
<cr>	Open (return key)
t	Open in new tab
p	Preview
%	create new file
d	create new directory
c	Set browsing directory as current directory.
d	Create directory
qf	Show file info
i	Toggle between display modes
s	Sort
r	Reverse
Ctrl-L	Refresh

" Query

qb	List book marks
qf	Show file info

" Go

gb	Go to previous bookmarked directory
1gb	Go to bookmark 1
2gb	Go to bookmark 2

" Mark

mb	Bookmark current directory
mf	Mark file
mr	Mark using regex
mu	Unmark all files
md	Diff the marked files
mt	Mark directory
mm	Move marked file to marked directory
mc	Copy

Commands

:e .	Start file explorer from here

==========